



Rate limits for the REST API

Learn about REST API rate limits, how to avoid exceeding them, and what to do if you do exceed them.

[Copy as Markdown](#)

In this article

[About primary rate limits](#)[About secondary rate limits](#)[Checking the status of your rate limit](#)[Exceeding the rate limit](#)[Staying under the rate limit](#)[Getting a higher rate limit](#)

About primary rate limits

GitHub limits the number of REST API requests that you can make within a specific amount of time. This limit helps prevent abuse and denial-of-service attacks, and ensures that the API remains available for all users.

Some endpoints, like the search endpoints, have more restrictive limits. For more information about these endpoints, see [REST API endpoints for rate limits](#). The GraphQL API also has a separate primary rate limit. See [Rate limits and query limits for the GraphQL API](#).

In general, you can calculate your primary rate limit for the REST API based on your method of authentication, as described below.

Primary rate limit for unauthenticated users

You can make unauthenticated requests if you are only fetching public data. Unauthenticated requests are associated with the originating IP address, not with the user or application that made the request.

The primary rate limit for unauthenticated requests is 60 requests per hour.

Primary rate limit for authenticated users

You can use a personal access token to make API requests. Additionally, you can authorize a GitHub App or OAuth app, which can then make API requests on your behalf.

All of these requests count towards your personal rate limit of 5,000 requests per hour. Requests made on your behalf by a GitHub App that is owned by a GitHub Enterprise Cloud organization have a higher rate limit of 15,000 requests per hour. Similarly, requests made on your behalf by a OAuth app that is owned or approved by a GitHub Enterprise Cloud organization have a higher rate limit of 15,000 requests per hour if you are a member of the GitHub Enterprise Cloud organization. However, requests made by a higher-limit app reduce the remaining budget available for lower-limit authentication methods. For example, if an app with a 15,000 request limit makes 10,000 requests on your behalf, you will have exhausted the 5,000 request budget for your personal access tokens, even though the app has 5,000 requests remaining.

Primary rate limit for Git LFS access

API requests are required when you upload or download Git LFS content. These count towards a separate rate limiting bucket with a limit of 300 requests per minute for unauthenticated requests and 3,000 requests per minute for authenticated requests.

Git LFS uses a batch API which processes 100 Git LFS objects per API request by default. That means unauthenticated users can download 30,000 Git LFS objects per minute and authenticated users can upload/download 300,000 Git LFS objects per minute.

Primary rate limit for GitHub App installations

GitHub Apps authenticating with an installation access token use the installation's minimum rate limit of 5,000 requests per hour. If the installation is on a GitHub Enterprise Cloud organization, the installation has a rate limit of 15,000 requests per hour.

For installations that are not on a GitHub Enterprise Cloud organization, the rate limit for the installation will scale with the number of users and repositories. Installations that have more than 20 repositories receive another 50 requests per hour for each repository. Installations that are on an organization that have more than 20 users receive another 50 requests per hour for each user. The rate limit cannot increase beyond 12,500 requests per hour.

Primary rate limits for GitHub App user access tokens (as opposed to installation access tokens) are dictated by the primary rate limits for the authenticated user. This rate limit is combined with any requests that another GitHub App or OAuth app makes on that user's behalf and any requests that the user makes with a personal access token. For more information, see [Rate limits for the REST API](#).

Primary rate limit for OAuth apps

Primary rate limits for OAuth access tokens generated by a OAuth app are dictated by the primary rate limits for authenticated users. This rate limit is combined with any requests that another GitHub App or OAuth app makes on that user's behalf and any requests that the user makes with a personal access token. See [Primary rate limit for authenticated users](#).

OAuth apps can also use their client ID and client secret to fetch public data. For example:

```
curl -u YOUR_CLIENT_ID:YOUR_CLIENT_SECRET -I https://api.github.com/meta
```

For these requests, the rate limit is 5,000 requests per hour per OAuth app. If the app is owned by a GitHub Enterprise Cloud organization, the rate limit is 15,000 requests per hour.

Note

Never include your app's client secret in client-side code or in code that runs on a user device. The client secret can be used to generate OAuth access tokens for users who have authorized your app, so you should always keep the client secret secure.

Primary rate limit for `GITHUB_TOKEN` in GitHub Actions

You can use the built-in `GITHUB_TOKEN` to authenticate requests in GitHub Actions workflows. See [Use GITHUB_TOKEN for authentication in workflows](#).

The rate limit for `GITHUB_TOKEN` is 1,000 requests per hour per repository. For requests to resources that belong to a GitHub Enterprise Cloud account, the limit is 15,000 requests per hour per repository.

About secondary rate limits

In addition to primary rate limits, GitHub enforces secondary rate limits in order to prevent abuse and keep the API available for all users.

You may encounter a secondary rate limit if you:

- *Make too many concurrent requests.* No more than 100 concurrent requests are allowed. This limit is shared across the REST API and GraphQL API.
- *Make too many requests to a single endpoint per minute.* No more than 900 points per minute are allowed for REST API endpoints, and no more than 2,000 points per minute are allowed for the GraphQL API endpoint. For more information about points, see [Calculating points for the secondary rate limit](#).
- *Make too many requests per minute.* No more than 90 seconds of CPU time per 60 seconds of real time is allowed. No more than 60 seconds of this CPU time may be for the GraphQL API. You can roughly estimate the CPU time by measuring the total response time for your API requests.
- *Make too many requests that consume excessive compute resources in a short period of time.*
- *Create too much content on GitHub in a short amount of time.* In general, no more than 80 content-generating requests per minute and no more than 500 content-generating requests per hour are allowed. Some endpoints have lower content creation limits. Content creation limits include actions taken on the GitHub web interface as well as via the REST API and GraphQL API.
- *Make too many OAuth access token requests in a short period of time.* No more than 2,000 OAuth access token requests per hour are allowed for GitHub Apps and OAuth apps.

These secondary rate limits are subject to change without notice. You may also encounter a secondary rate limit for undisclosed reasons.

Calculating points for the secondary rate limit

Some secondary rate limits are determined by the point values of requests. For GraphQL requests, these point values are separate from the point value calculations for the primary rate limit.

Request	Points
GraphQL requests without mutations	1
GraphQL requests with mutations	5
Most REST API GET , HEAD , and OPTIONS requests	1
Most REST API POST , PATCH , PUT , or DELETE requests	5

Some REST API endpoints have a different point cost that is not shared publicly.

Checking the status of your rate limit

You can use the headers that are sent with each response to determine the current status of your primary rate limit.

Header name	Description
<code>x-ratelimit-limit</code>	The maximum number of requests that you can make per hour
<code>x-ratelimit-remaining</code>	The number of requests remaining in the current rate limit window
<code>x-ratelimit-used</code>	The number of requests you have made in the current rate limit window
<code>x-ratelimit-reset</code>	The time at which the current rate limit window resets, in UTC epoch seconds
<code>x-ratelimit-resource</code>	The rate limit resource that the request counted against. For more information about the different resources, see REST API endpoints for rate limits .

You can also call the `GET /rate_limit` endpoint to check your rate limit. Calling this endpoint does not count against your primary rate limit, but it can count against your secondary rate limit. See [REST API endpoints for rate limits](#). When possible, you should use the rate limit response headers instead of calling the API to check your rate limit.

There is not a way to check the status of your secondary rate limit.

Exceeding the rate limit

If you exceed your primary rate limit, you will receive a `403` or `429` response, and the `x-ratelimit-remaining` header will be `0`. You should not retry your request until after the time specified by the `x-ratelimit-reset` header.

If you exceed a secondary rate limit, you will receive a `403` or `429` response and an error message that indicates that you exceeded a secondary rate limit. If the `retry-after` response header is present, you should not retry your request until after that many seconds has elapsed. If the `x-ratelimit-remaining` header is `0`, you should not retry your request until after the time, in UTC epoch seconds, specified by the `x-ratelimit-reset` header. Otherwise, wait for at least one minute before retrying. If your request continues to fail due to a

secondary rate limit, wait for an exponentially increasing amount of time between retries, and throw an error after a specific number of retries.

Continuing to make requests while you are rate limited may result in the banning of your integration.

Staying under the rate limit

You should follow best practices to help you stay under the rate limits. See [Best practices for using the REST API](#).

Getting a higher rate limit

If you want a higher primary rate limit, consider making authenticated requests instead of unauthenticated requests. Authenticated requests have a significantly higher rate limit than unauthenticated requests.

If you are using a personal access token for automation in your organization, consider whether a GitHub App will work instead. The rate limit for GitHub Apps using an installation access token scales with the number of repositories and number of organization users. See [About creating GitHub Apps](#).

If you are using GitHub Apps or OAuth apps, consider upgrading to GitHub Enterprise Cloud. GitHub Apps or OAuth apps have higher rate limits for organizations that use GitHub Enterprise Cloud.

Legal

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Status](#) [Pricing](#) [Expert services](#) [Blog](#)